

in. [Listing 2-2](#) contains the whole function.

```
import random

def fillsquare(square, entry_i, entry_j, howfull):
    while(sum(math.isnan(i) for row in square for i in row) > howfull):
        where_we_can_go = []

        if(entry_i < (n - 1) and entry_j < (n - 1)):
            where_we_can_go.append('down_right')
        if(entry_i < (n - 1) and entry_j > 0):
            where_we_can_go.append('down_left')
        if(entry_i > 0 and entry_j < (n - 1)):
            where_we_can_go.append('up_right')
        if(entry_i > 0 and entry_j > 0):
            where_we_can_go.append('up_left')

        where_to_go = random.choice(where_we_can_go)
        if(where_to_go == 'up_right'):
            new_entry_i = entry_i - 1
            new_entry_j = entry_j + 1
            square[new_entry_i][new_entry_j] = rule1(square[entry_i]
[entry_j], n, True)

            if(where_to_go == 'down_left'):
                new_entry_i = entry_i + 1
                new_entry_j = entry_j - 1
                square[new_entry_i][new_entry_j] = rule1(square[entry_i]
[entry_j], n, False)

            if(where_to_go == 'up_left' and (entry_i + entry_j) != (n)):
                new_entry_i = entry_i - 1
                new_entry_j = entry_j - 1
                square[new_entry_i][new_entry_j] = rule2(square[entry_i]
[entry_j], n, True)

            if(where_to_go == 'down_right' and (entry_i + entry_j) != (n-2)):
                new_entry_i = entry_i + 1
                new_entry_j = entry_j + 1
                square[new_entry_i][new_entry_j] = rule2(square[entry_i]
[entry_j], n, False)

            if(where_to_go == 'up_left' and (entry_i + entry_j) == (n)):
                new_entry_i = entry_i - 1
                new_entry_j = entry_j - 1
```